

Automação e Documentação de APIs



Conteudista: Prof. Esp. Jônatas Carvalho

Revisão Textual: Esp. Anna Carolina Guimarães

Objetivo da Unidade:

- Conhecer as ferramentas de automação e documentação de APIs de forma eficiente.

 **Material Teórico**

 **Referências**



Material Teórico

Documentação de Projetos

Projetos independentes de seu tamanho devem possuir documentação. Equipes não nascem junto com um projeto e permanecem imutáveis até o seu fim, muito pelo contrário; com o mercado aquecido, é muito comum que equipes mudem sua configuração constantemente. Esse é apenas um dos motivos que justifica a criação de uma documentação para o projeto.

Documentações são essenciais para deixar em evidência informações relevantes e importantes de um projeto, pois essas informações garantem o alinhamento das partes interessadas em um projeto (clientes, desenvolvedores, coordenadores de projeto etc.).

Os propósitos de uma documentação devem estar firmados em comunicar informações de forma clara, facilitando o entendimento entre os envolvidos no projeto. Registrar decisões tomadas em forma de marcos do projeto a fim de mostrar mudanças e evoluções, e auxiliar na manutenção e suporte do produto.

Existem diferentes tipos de documentação, como documentação de requisitos, documentação de processos, documentação de usuário, documentação de testes e documentação técnica. Assim como existem vários tipos de documentação, também existem vários tipos de ferramentas para criação de documentação, as mais utilizadas são Swagger, Confluence e as que são criadas a base da linguagem *Markdown*. Em nosso material, vamos criar uma documentação técnica utilizando o Swagger.

Swagger

Swagger é um conjunto de serviços *open source* (de código aberto que podem ser usados por qualquer um gratuitamente) que facilitam a criação de documentações. Em 2015, o *Swagger Specification* foi doado para a *OpenAPI Initiative* assim passando a se chamar *OpenAPI Specification (OAS)*, porém as ferramentas que utilizam essa especificação ainda são conhecidas pelo nome Swagger.

Os principais componentes do Swagger são:

- **Swagger UI:** é uma interface interativa para documentação de Apis na qual permite que desenvolvedores possam testar diretamente do navegador sua aplicação sem a necessidade de outras ferramentas;
- **Swagger Editor:** é um editor on-line, como se fosse uma ide que permite a criação de arquivos, como yaml ou json, fornecendo a geração automática da documentação;
- **Swagger Codegen:** é uma ferramenta que cria códigos em várias linguagens de programação baseados na especificação *OpenAPI*;
- **SwaggerHub:** é uma plataforma colaborativa que é muito útil quando equipes necessitam gerenciar e compartilhar especificações.

Utilizar o Swagger como ferramenta para criação de documentação nos traz muitos benefícios, pois acelera o desenvolvimento, sempre oferecendo uma visão clara de como deve ser o funcionamento da aplicação, exibindo de forma clara as entradas e saídas que um *endpoint* deve receber e devolver, isso por si só já reduz em muito erros de comunicação dentro do projeto. Outros benefícios que podem ser citados são a padronização que automaticamente seguindo a *OpenAPI Specification* deixa mais uniforme as informações e a documentação interativa, na qual consumidores que utilizarão a aplicação podem visualizar *endpoint*, métodos, parâmetros e exemplos.

Funcionamento e Exemplo Prático com Swagger

Basicamente o Swagger funciona da seguinte maneira, primeiro devemos criar um arquivo podendo ser no formato yaml ou json. Nesse arquivo, vamos descrever os *endpoints* da API, métodos, parâmetros e respostas que a aplicação deve fornecer. Após criarmos esse documento, podemos usar o Swagger Codegen para gerar a documentação. Após isso, nossa documentação está pronta, e consumidores e desenvolvedores podem utilizar a documentação como material de apoio.

Iremos criar a documentação para a aplicação que desenvolvemos na Unidade 2, lembrando que ela possui três classes: `app.js`, `index.js` e `validateDecimal.js`. A aplicação roda no endereço `localhost:3000`, possuindo o *endpoint* `/to-binary/XXX`, na qual o XXX é um número decimal que o usuário deve informar, com isso, a url completa para realizar uma requisição seria por exemplo: `localhost:3000/to-binary/25`.

Crie um arquivo com a extensão `.yaml`, por exemplo: `documentacao.yaml` e insira o código abaixo dentro do seu arquivo.

```
#CABEÇALHO
openapi: 3.0.0
info:
  title: Conversor de Decimal para Binário API
  version: 1.0.0
  description: |
    Uma API que converte um número decimal para sua representação binária.

  ## Endpoints:
  - `/to-binary/{decimal}`: Converte um número decimal para binário.

#SERVIDORES
servers:
  - url: http://localhost:3000
    description: Servidor de desenvolvimento local
paths:
  /to-binary/{decimal}:
```

```
get:
  summary: Converter decimal para binário
  description: Converte um número decimal fornecido como parâmetro
de caminho para sua representação binária.

#PARAMETROS
parameters:
  - name: decimal
in: path
required: true
description: O número decimal a ser convertido para binário.
schema:
  type: integer
  example: 10

#RESPOSTAS
responses:
  '200':
description: Resposta bem-sucedida com os valores decimal e binário
content:
  application/json:
    schema:
      type: object
      properties:
        decimal:
          type: integer
          example: 10
        binary:
          type: string
          example: "1010"
  '400':
description: Número decimal inválido fornecido.
content:
  application/json:
    schema:
      type: object
      properties:
        error:
          type: string
          example: "Número decimal inválido"
```

A documentação que acabamos de criar vai gerar o seguinte resultado, exibido na seguinte imagem:



Figura 1 – Documentação criada pelo Swagger

Fonte: Acervo do conteudista

#ParaTodosVerem: a imagem exibe a documentação criada pelo Swagger. Na parte superior, temos o título junto com a versão, mas abaixo temos os *endpoints* junto com a url completa. Na parte inferior da página, temos o *GET* do endereço de requisição. Fim da descrição.

Vamos aos principais pontos de nossa documentação. Para facilitar, foram inseridos alguns comentários para que a explicação fique melhor; esses comentários começam com apenas uma #. No início do arquivo, começamos com o comentário #CABEÇALHO. Nesta sessão, especificamos a versão do padrão *OpenAPI* que utilizaremos a versão 3.0.0 e, então, temos a chave *info* que irá conter as informações gerais sobre a aplicação. Nessa chave, vamos ter o título da aplicação, na qual devemos definir através do problema em que a aplicação procura resolver. No nosso caso, Conversor de

Decimal para Binário. Após isso, definimos a versão da aplicação e, também, uma breve descrição das funcionalidades.

No comentário #SERVIDORES, teremos a definição do endereço principal do servidor, os caminhos que a aplicação possui, os métodos que ela implementa, que em nosso caso é apenas um *endpoint* com o método *GET* e uma descrição que pode ser opcional, que, normalmente, informamos qual é o ambiente de desenvolvimento (*local*, *qa*, *sandbox*, *prod*).

Já em #PARAMETRO vamos definir o nome do parâmetro que devemos passar para nossa aplicação, que nesse caso é um decimal e passaremos ele diretamente na URL da requisição. Além disso, o parâmetro é obrigatório, por isso, na chave *required*, temos *true*, e, em *schema*, informamos que o dado esperado é um número inteiro e damos um exemplo de uso.

Por último, temos a sessão de #RESPOSTAS. Nela temos os retornos que a aplicação nos dá. Nesse caso são dois retornos, *http status code* 200 e 400. O código 200 tem a função de retornar para o usuário que a requisição foi realizada com sucesso, já o código 400 quer dizer que algo de errado aconteceu. Aqui, provavelmente, o usuário informou um número inválido. Nesta seção, mostramos um exemplo do json que deve ser enviado no *body* da requisição e a resposta que a aplicação vai fornecer segundo aquela requisição.

GET

/to-binary/{decimal}

Converter decimal para binário

^

Converte um número decimal fornecido como parâmetro de caminho para sua representação binária.

Parameters

Try it out

Name	Description
decimal ★ required integer (path)	O número decimal a ser convertido para binário. 10

Responses

Code	Description	Links
200	Resposta bem-sucedida com os valores decimal e binário. Media type application/json Controls Accept header. Example Value Schema <pre>{ "decimal": 10, "binary": "1010"}</pre>	No links
400	Número decimal inválido fornecido. Media type application/json Example Value Schema <pre>{ "error": "Número decimal inválido"}</pre>	No links

Figura 2 – Requisições na documentação do Swagger

Fonte: Acervo do conteudista

#ParaTodosVerem: na imagem, vemos as duas requisições da documentação, exibindo o *status code* de saída e os json que recebemos como resposta nas duas requisições. Fim da descrição.

Postman

Uma das ferramentas mais utilizadas por desenvolvedores é o Postman. Ela é muito útil para o desenvolvimento, o teste e a documentação de aplicações. Através do Postman, é possível criar requisições do tipo http, como: *get*, *post*, *put* e *delete*. Através dessas requisições, podemos enviar e receber dados a fim de validar se as aplicações estão se comportando como deveriam.

Postman possui uma interface muito intuitiva, que facilita seu uso. Além disso, é multiplataforma, podendo rodar em diversos sistemas operacionais, como MacOS, Windows e Linux. Além de ser uma ferramenta que pode ser utilizada de forma gratuita, possui uma ótima integração com o Swagger e o OpenAPI.

Além de ser muito útil na automatização de testes, pois aceita a criação de *scripts* em Javascript, que podem validar respostas e realizar a simulação de cenários e testes automatizados. Podemos organizar nossas requisições em coleções e, também, configurá-las para trabalharem em diversos ambientes diferentes, como: *sandbox*, produção e qa.

A ferramenta em questão é tão rica que podemos empregá-la em monitoramentos de desempenho de APIs, verificando se estas estão funcionando de forma correta em intervalos definidos.

Pelos fatos acima apresentados, podemos ver que o Postman é uma ótima ferramenta para o trabalho, podendo ajudar equipes a compartilharem coleções, criar documentação e aprimorar testes.

Criando Requisições com o Postman

Trabalhar com Postman e Swagger é essencial quando o assunto é documentação e teste de aplicações. Vamos utilizar a documentação que criamos anteriormente para exportar todos os *endpoints* da nossa aplicação.

Com o Postman aberto, clique no botão *Import* no menu à esquerda:

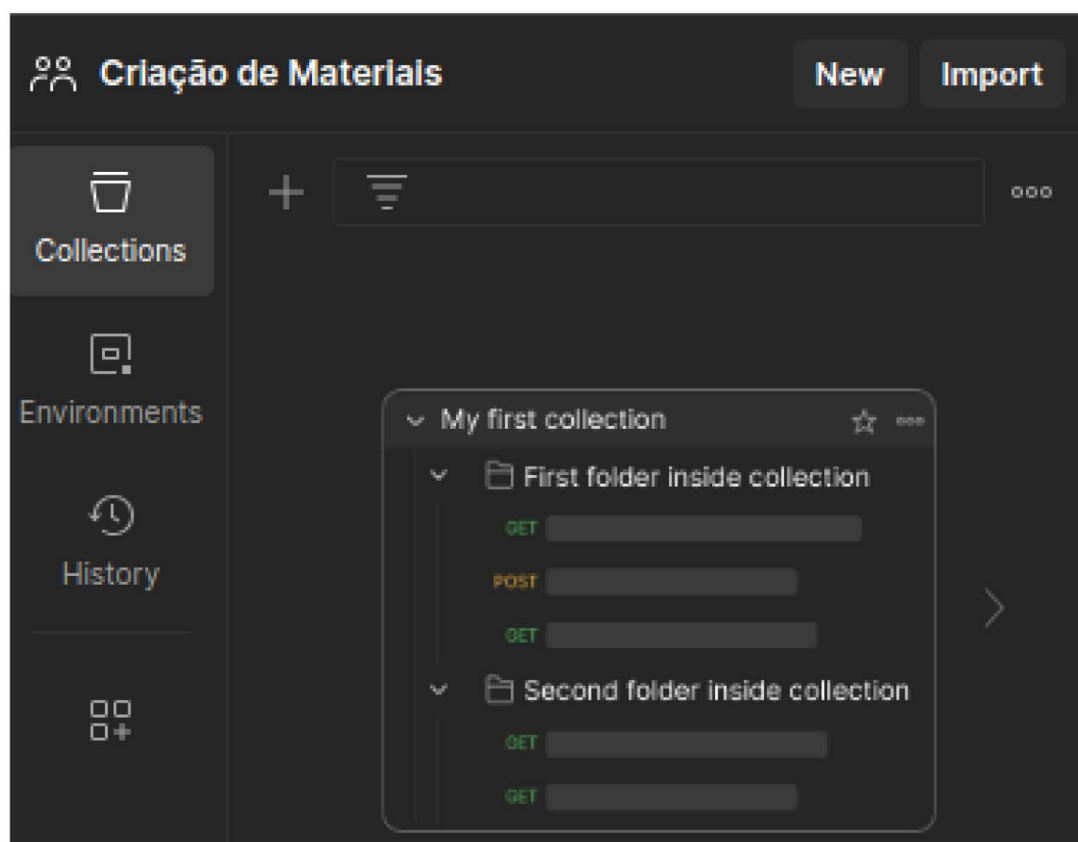


Figura 3 – Menu da parte esquerda do Postman

Fonte: Acervo do conteudista

#ParaTodosVerem: a imagem exibe o menu esquerdo do Postman. Na parte superior, temos o botão *New*, que serve para criar novas requisições, e o botão de *Import* para importar requisições. Fim da descrição.

Após clicar no botão *Import*, uma janela vai abrir, e, então, clique em *files* e procure o arquivo da documentação que criamos anteriormente, ou arraste o arquivo para esta janela:

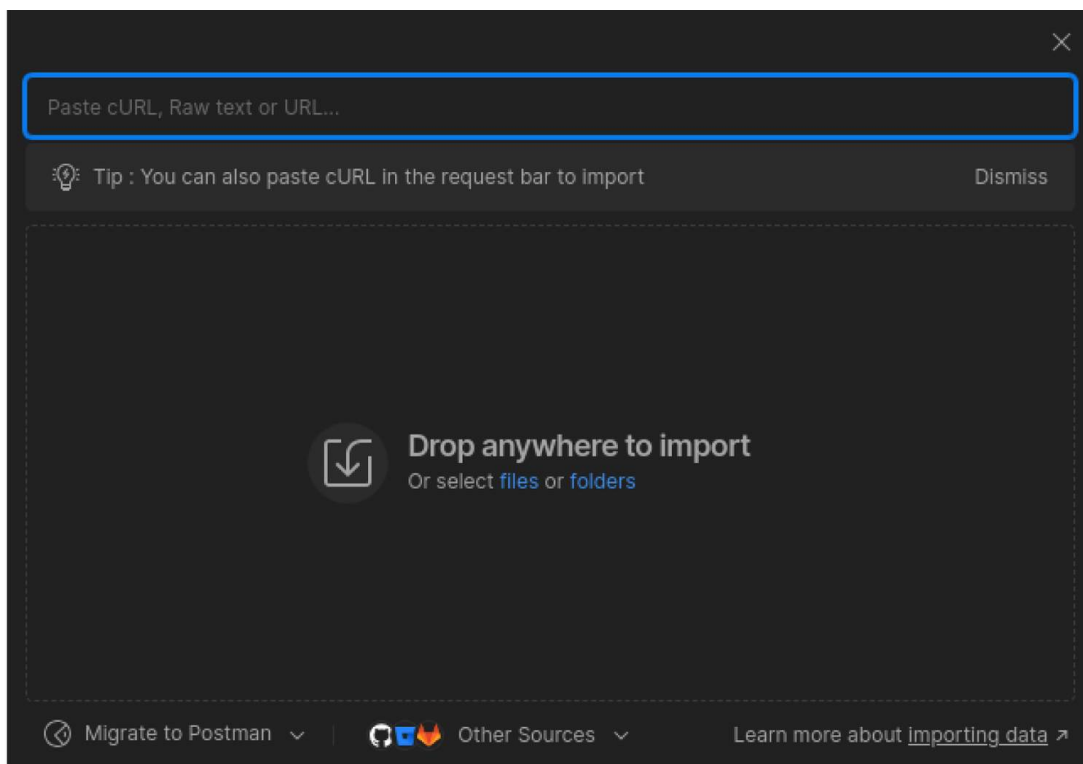


Figura 4 – Janela de importação de *collections*

Fonte: Acervo do conteudista

#ParaTodosVerem: a imagem exibe a tela de importação de *collections* e requisições. Na parte superior, há uma barra de pesquisa e, na parte inferior, podemos dropar os arquivos. Fim da descrição.

No menu lateral esquerdo, aparecerá a coleção que foi criada em nossa documentação do *Swagger*, com os dois *endpoints*, o primeiro com a resposta bem-sucedida, e o segundo com uma requisição inválida, como se segue nas próximas duas imagens:

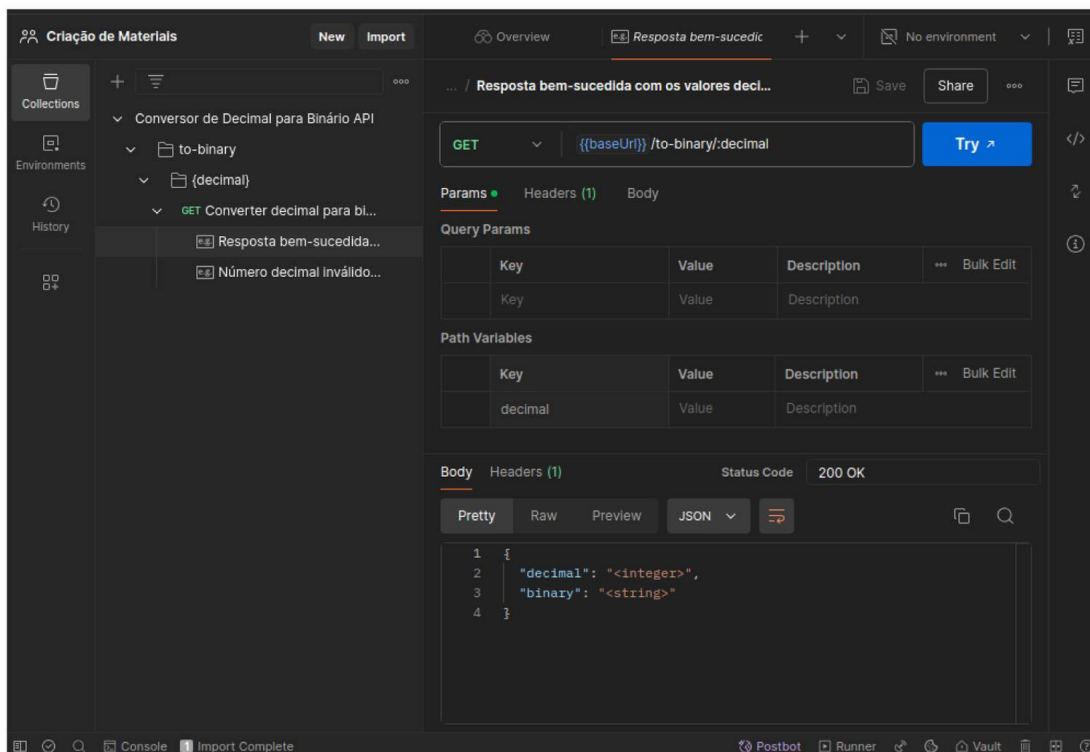


Figura 5 – Janela do Postman

Fonte: Acervo do conteudista

#ParaTodosVerem: a imagem exibe a janela inteira do Postman. Do lado esquerdo, mostra nossas requisições criadas através do Swagger. Do lado direito, temos a requisição que é do tipo *GET*, a URL e um botão *Try* para disparar a requisição. Mais abaixo, temos o JSON que recebemos como retorno após disparar a requisição. Fim da descrição.

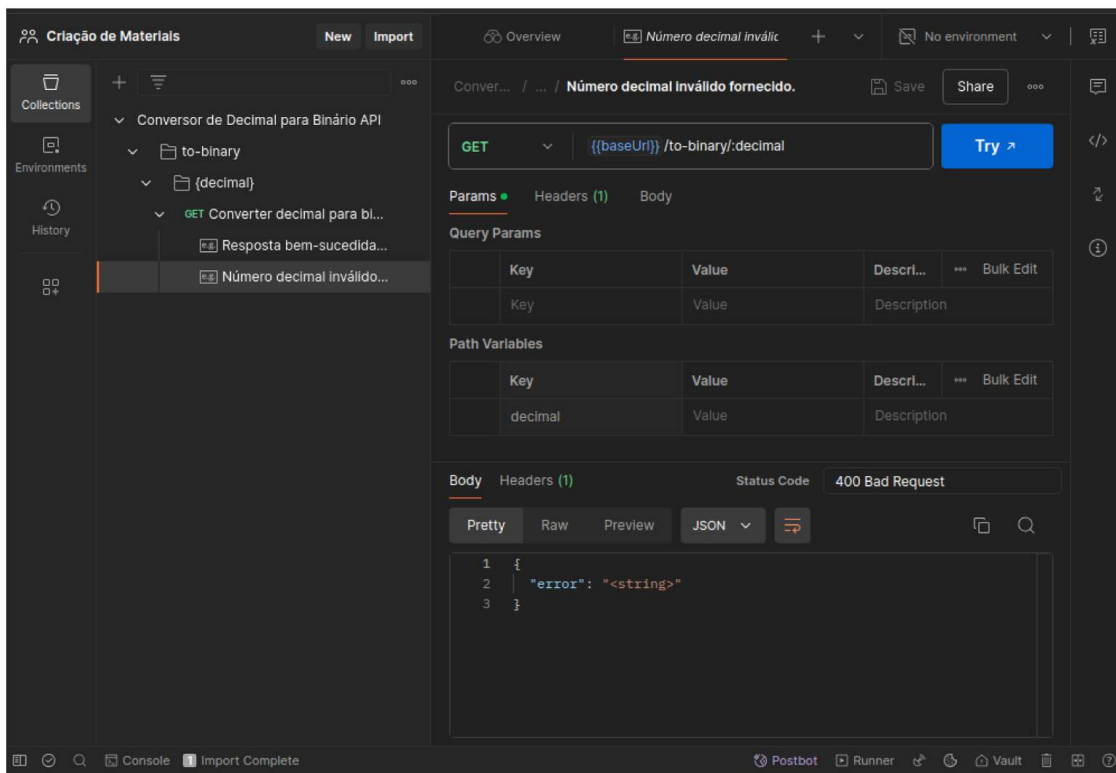


Figura 6 – Janela do Postman

Fonte: Acervo do conteudista

#ParaTodosVerem: a imagem exhibe a janela inteira do Postman. Do lado esquerdo, mostra nossas requisições criadas através do Swagger. Do lado direito, temos a requisição que é do tipo *GET*, a URL e um botão *Try* para disparar a requisição. Mais abaixo tem o JSON que recebemos como retorno após disparar a requisição. Fim da descrição.

É notável que trabalhar com esse conjunto de ferramentas ajuda muito na questão de organização das coleções e dos *endpoints*, isso sem mencionar a facilidade e a economia de tempo. Então vale muito a pena utilizar Swagger e Postman juntos.

API Gateway

Quando falamos de arquiteturas modernas de software, *API Gateways* possuem um papel muito importante nas questões de gerenciamento, proteção e controle de sistemas distribuídos. Principalmente em arquiteturas em que os microserviços

compõem a maioria da aplicação, *API Gateways* se tornam indispensáveis, pois há muito complexidade na questão de *endpoints*, interações e protocolos e, nesses casos, ele funciona como um mediador com clientes e outras aplicações que fazem uso da API.

Uma *API Gateway* centraliza a permissão de acesso às APIs e, também, possui diversas funcionalidades que são essenciais, como limitação de taxa, balanceamento de carga e manipulação de dados. Isso é ótimo, pois podemos ter uma camada de abstração que deixará oculta toda a complexidade dos serviços internos.

No mercado, encontramos diversas soluções quando o assunto é *API Gateway*. O Kong é uma das mais populares e uma das mais robustas e, por isso, possui grande adoção por empresas de portes diferentes e projetos diferentes. Kong, além de ser *open source*, possui um grau elevado de personalização, além de ser muito rápido é muito eficiente e poderoso.

Kong é muito personalizável, pois além de possuir muitas funcionalidades nativas, ele pode ser melhorado com *plugins* que são criados por uma grande comunidade de desenvolvedores que dá apoio a utilização desta ferramenta. Além desta questão, o Kong possui uma boa integração com ferramentas como o Prometheus e o Grafana para que a análise dos dados, comportamentos e desempenho das aplicações possam ser exibidas em uma interface amigável.

Além de ser muito utilizado para a gestão de aplicações, sejam elas internas ou públicas, o Kong proporciona uma ótima flexibilidade para pequenas equipes de desenvolvimento quanto para grandes corporações, trabalha muito bem em ambientes *multicloud* e híbridos.

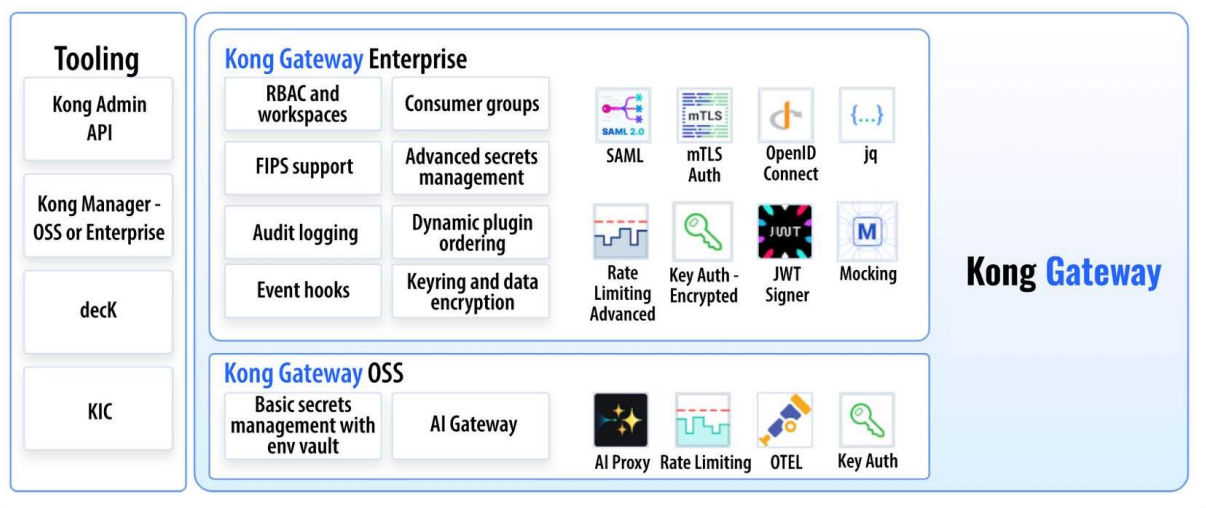


Figura 7 – Exemplo de Funcionamento do Kong

Fonte: Reprodução

#ParaTodosVerem: a imagem exibe um diagrama dos principais recursos do Kong Gateway. No lado esquerdo, temos ferramentas nativas do Kong, como Kong Manager OSS, deckK, KIC e, no lado direito, temos plugins como SAML, OpenID Connect, Mocking, Rate Limiting etc.

Por fim, essa ferramenta se destaca muito pelo seu desempenho, por sua infinidade de funcionalidades, uma comunidade extremamente ativa e possui uma documentação abrangente. Em resumo, o Kong é uma solução muito versátil, possui grande potencial de escalabilidade em qualquer ambiente.

Docker

Como em nossas próximas aulas vamos utilizar diversas ferramentas, acho importante aprendermos um pouco sobre *Docker* e *containers*.

Docker é uma plataforma de containerização que empacota aplicações em *containers* combinando as dependências e as bibliotecas com sistemas operacionais. Utilizar *container* facilita muito a vida, pois podemos rodar os mesmos ambientes em diferentes locais, sejam máquinas ou sistemas operacionais. O *Docker* fornece às

equipes de desenvolvimento uma plataforma que equipara as ferramentas para todos, podendo ser obtido os mesmos resultados para todo o desenvolvimento.

Máquinas virtuais comumente são confundidas com os princípios de containerização, porém são conceitos totalmente distintos. Uma máquina virtual emula toda a estrutura de um sistema operacional, enquanto o *Docker* carrega consigo apenas ferramentas essenciais de um sistema operacional para poder emular aplicações. Enquanto máquinas virtuais podem ter o tamanho que ocupa gigabytes, os *containers Docker* podem ocupar poucos megabytes.

Normalmente, quando vamos criar uma infraestrutura com diversas aplicações rodando através de um *container*, utilizaremos a abordagem de criar um *Docker Compose*, que é um arquivo com a extensão `yml` ou `yaml` que ficará encarregado de criar os *containers*, as redes e todas as dependências utilizadas pelas aplicações que nele estarão.

Site

Docker Desktop

Para instalar o *Docker*, basta entrar no site e, então, fazer o download do executável e instalá-lo. É uma instalação simples, basta avançar até o final.

Clique no botão para conferir o conteúdo.

ACESSE

Em nossas próximas Unidades, o *Docker* vai ser muito útil, pois não vamos instalar nada em nossa máquina, vamos criar um arquivo *docker-compose* que vai executar todas as nossas ferramentas sem a necessidade de instalação.



Referências

DE, B. **API management: an architect's guide to developing and managing APIs for your organization**. 1. ed. Sebastopol: O'Reilly Media, 2017.

HUMBLE, J.; FARLEY, D. **Continuous delivery: reliable software releases through build, test, and deployment automation**. 1. ed. Upper Saddle River: Addison-Wesley, 2010.

JIN, B.; SAHNI, S.; SHEVAT, A. **Designing web APIs: building APIs that developers love**. 1. ed. Sebastopol: O'Reilly Media, 2018.

MATTHIAS, K.; KANE, S. P. **Docker: up & running: shipping reliable containers in production**. 2. ed. Sebastopol: O'Reilly Media, 2020.